



ZeroGS 训练报告

github仓库: <https://github.com/knotraveller/3D-PJ>

1. 实验概述

本报告基于当前工程中的 `configs/zerogs_train.yaml` , 以及 `outputs/zerogs_train` 下已经生成的训练日志、验证日志、Gaussian 统计和性能统计整理。

实验目标: 给定同一物体的 7 个带位姿视角图像, 前向预测一组三维 Gaussian 参数, 再通过 `gsplat` 渲染回多视角 RGB/alpha 图像, 并用重建损失进行监督训练。虽然形式上仿佛只是重建了图像, 但实际上生成了 3d Gaussian 资产

当前主要启动命令:

```
.\scripts\run_train.ps1 --config .\configs\zerogs_train.yaml
```

主要输出目录:

内容	路径
训练日志	<code>outputs/zerogs_train/logs/train_log.jsonl</code>
验证日志	<code>outputs/zerogs_train/validate/logs/validate_log.jsonl</code>
TensorBoard	<code>outputs/zerogs_train/tensorboard</code>
可视化图像	<code>outputs/zerogs_train/visuals</code>
checkpoint	<code>outputs/zerogs_train/checkpoints</code>
Gaussian/性能统计	<code>outputs/zerogs_train/stats</code>
loss 曲线	<code>outputs/zerogs_train/plots/loss_curve.png</code>

2. 模型结构简要介绍

2.1 输入与输出

数据集每个样本包含 7 个视角。训练时将 RGB 图像和相机射线编码拼接为模型输入：

- RGB: 3 通道。
- Plucker ray embedding: 6 通道。
- 总输入: $[B, 7, 9, 256, 256]$ 。

ZeroGSUNet 输出 raw Gaussian map:

- raw 输出: $[B, 7, 16, 64, 64]$ 。
- Gaussian map: $[B, 7, 14, 64, 64]$ 。
- 展平后的 Gaussian 列表: $[B, 28672, 14]$ ，其中 $28672 = 7 * 64 * 64$ 。

每个 Gaussian 的 14 维含义为：

```
[x, y, z, opacity, sx, sy, sz, qw, qx, qy, qz, r, g, b]
```

2.2 ZeroGSUNet 主干设计

这是我独立性最大的部分。为了兼顾性能和效果，我采用了 U-Net 和 view attention

模型是一个 LGM 风格的多视角 U-Net。实现上将 batch 和 view 合并，用共享 2D 卷积处理每个视角，再在中高层特征处加入跨视角注意力。

主要结构如下：

- Stem: $9 \rightarrow \text{base_channels}$ ，当前 $\text{base_channels}=64$ 。
- Encoder: 多层 ResBlock + Downsample，分辨率从 $256 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 16$ 。
- View attention: 在中间尺度特征上，逐 feature 做跨视角信息交互。（为了应付算力不够，而使用的一种简单attention）
- Global attention: 在 bottleneck 处做全局信息交互。（由于算力不够，实验中未开启。可选）
- Decoder: 从 $16 \rightarrow 32 \rightarrow 64$ 上采样，并拼接 skip connection。
- Gaussian head: 将 64×64 特征映射为 16 通道 raw Gaussian 参数。

当前性能统计中记录的模型可训练参数量为 56,189,456 , 模型参数显存约 214.35 MB 。

2.3 Gaussian 参数后处理

`raw_to_gaussians` 会把网络 raw 输出转换为可渲染的 3D Gaussian:

- `depth`: 由 sigmoid 映射到 `[2.5, 5.5]` 。
- `offset`: 经过 `tanh` 后乘以 `offset_scale=0.05` 。
- `scale`: 由 `softplus` 得到, 并 clamp 到稳定范围。
- `quaternion`: 归一化为旋转四元数。
- `opacity`: 使用 `sigmoid(opacity_raw - opacity_bias)`, 当前 `opacity_bias=4.0`, 用于避免初始 alpha 过度饱和。
- `RGB`: 使用 sigmoid 映射到 `[0, 1]` 。

相机坐标采用 Blender 风格生成世界坐标; 渲染前在 `renderer` 中将 view matrix 转换为 `gsplat` 期望的 OpenCV 风格相机坐标。

2.4 渲染与损失

渲染器为 `GSplatRenderer`, 内部调用 `gsplat.rendering.rasterization`, 输出:

- `pred_rgb`: `[B, 7, 3, 256, 256]`
- `pred_alpha`: `[B, 7, 1, 256, 256]`

损失函数为 `ReconstructionLoss`:

```
total_loss = 1.0 * rgb_loss + 0.5 * mask_loss + 0.1 * lpips_loss
```

其中:

- `rgb_loss`: 默认只在 GT alpha 区域内计算 RGB L1。
- `mask_loss`: 预测 alpha 与 GT alpha 的 L1。
- `lpips_loss`: VGG LPIPS 感知损失, 当前开启。

3. zerogs_train 参数设置

3.1 实验与数据

参数	当前值
<code>experiment.name</code>	<code>zerogs_train</code>
<code>output_dir</code>	<code>outputs/zerogs_train</code>
<code>seed</code>	<code>42</code>
<code>train_root</code>	<code>dataset/renderers_256</code>
<code>val_root</code>	<code>dataset/renderers_256</code>
<code>image_size</code>	<code>256</code>
<code>num_workers</code>	<code>4</code>
<code>max_train_samples</code>	<code>null</code>
<code>max_val_samples</code>	<code>null</code>

3.2 模型参数

参数	当前值	说明
<code>in_channels</code>	<code>9</code>	RGB 3 通道 + ray embedding 6 通道
<code>num_views</code>	<code>7</code>	每个样本 7 个视角
<code>image_size</code>	<code>256</code>	输入/渲染分辨率
<code>splat_size</code>	<code>64</code>	Gaussian map 分辨率
<code>base_channels</code>	<code>64</code>	U-Net 基础通道数
<code>depth_min / depth_max</code>	<code>2.5 / 5.5</code>	深度预测范围
<code>offset_scale</code>	<code>0.05</code>	Gaussian 中心偏移尺度

参数	当前值	说明
opacity_bias	4.0	降低初始 opacity 饱和风险

3.3 渲染参数

参数	当前值
renderer.image_size	256
background	white
near_plane / far_plane	0.01 / 100.0
radius_clip	0.0
eps2d	0.3
packed	true
tile_size	16
rasterize_mode	classic

3.4 损失参数

参数	当前值
lambda_rgb	1.0
lambda_mask	0.5
lambda_lpips	0.1
use_lpips	true
lpips_net	vgg
lpips_chunk_size	1
mask_rgb_loss	true

3.5 训练参数

参数	当前值
epochs	1000
batch_size	1
lr	1.0e-6
lr_min	0.0
weight_decay	0.01
amp	true
grad_clip	1.0
gradient_accumulation_steps	2
scheduler	plateau
epoch_visuals	true
save_every	10
val_every	2
overfit_one_batch	false

3.6 性能记录参数

参数	当前值	说明
enabled	true	开启性能统计
ema_momentum	0.8	EMA 使用 $0.8 * \text{old_ema} + 0.2 * \text{current}$
sync_cuda	true	CUDA 同步计时，时间更准确但会增加开销
write_every	100	每 100 step 写一次性能统计
system_sample_every	100	系统采样间隔

参数	当前值	说明
sample_system	false	当前不采样 CPU/进程内存
sample_gpu_utilization	false	当前不采样 GPU 利用率
profile_gpu_modules	false	当前不记录模块级峰值显存

因此当前性能日志主要包含阶段耗时、模型参数显存、损失模块参数显存和优化器状态显存；没有 CPU/GPU 利用率曲线。

4. 当前结果指标

以下指标来自当前 `outputs/zerogs_train` 中的日志。截至当前日志，训练和验证都记录到 epoch 412。

4.1 训练集指标

epoch 412 是当前训练日志中的最新 epoch，同时也是当前训练日志中的最佳 loss/PSNR epoch。

指标	mean	min	max
loss	0.049543	0.000001	0.108700
rgb_loss	0.028629	0.000000	0.068282
mask_loss	0.022913	0.000001	0.107584
lpips_loss	0.094581	0.000006	0.231445
psnr	25.6200	19.3014	80.0000

4.2 验证集指标

epoch 412 是当前验证日志中的最新 epoch，同时也是当前验证日志中的最佳 loss/PSNR epoch。

指标	mean	min	max
loss	0.049461	0.000001	0.108562
rgb_loss	0.028588	0.000000	0.068298
mask_loss	0.022887	0.000001	0.107463
lpips_loss	0.094291	0.000005	0.230931
psnr	25.6270	19.2966	80.0000

训练集和验证集的 loss/PSNR 非常接近，说明当前数据划分下二者分布高度一致。需要注意的是，当前 `train_root` 和 `val_root` 都指向 `dataset/renderers_256`，因此这些验证结果更适合作为 pipeline 自治性和重建质量观察，不应直接理解为严格独立测试集指标。

4.3 Gaussian 统计

最新 Gaussian 统计文件为：

```
outputs/zerogs_train/stats/gaussian_stats_epoch_0412.json
```

指标	min	mean	max
opacity	0.000000	0.171882	0.999023
scale	0.005000	0.013962	0.120659
xyz	-1.047852	0.011973	1.088867
rgb	0.014671	0.390557	0.971191

Gaussian 数量统计：

指标	数值
总 Gaussian 数	28672
opacity > 0.01	7335，约 25.58%
opacity > 0.05	6848，约 23.88%

4.4 性能统计

最新性能统计文件为：

```
outputs/zerogs_train/stats/performance_latest.json
```

当前快照记录于 epoch 412、global step 53972、phase 为 val 。

训练阶段耗时统计，单位为 ms：

阶段	EMA	mean	min	max	count
train/iteration_total	517.62	562.71	436.89	84688.48	26462
train/forward_total	198.23	235.30	174.76	81055.94	26462
train/model_forward	136.30	133.90	119.14	655.91	26462
train/render	1.58	1.68	1.04	577.34	26462
train/loss	57.79	96.89	51.45	80741.23	26462
train/backward	294.96	286.72	259.47	550.81	26462

验证阶段耗时统计，单位为 ms：

阶段	EMA	mean	min	max	count
val/iteration_total	842.27	876.91	256.60	81489.46	13231
val/forward_total	840.96	852.50	255.17	81487.80	13231
val/model_forward	767.65	752.77	182.62	1513.98	13231
val/render	1.39	1.75	1.07	11.21	13231
val/loss	69.68	95.25	65.49	80712.68	13231

静态显存/参数统计：

项目	数值
模型可训练参数	56,189,456
模型参数显存	214.35 MB
损失模块参数显存	56.14 MB
优化器状态显存	428.69 MB

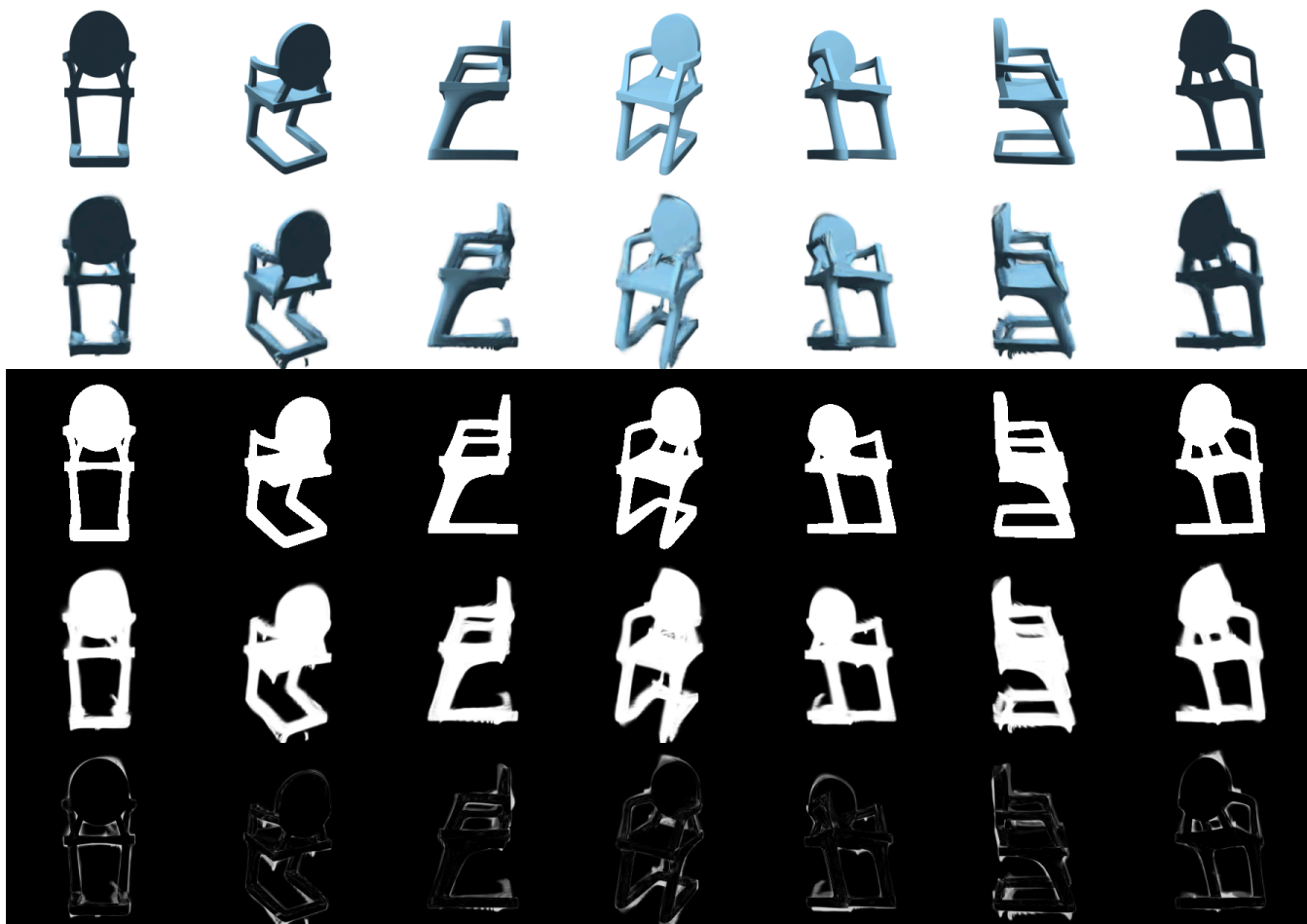
这里的显存统计主要覆盖参数和优化器状态，不包含每一步前向/反向传播的激活、中间 tensor、CUDA kernel workspace 和 PyTorch cache，因此不能直接等同于实际训练时 `nvidia-smi` 看到的总显存占用。

5. 结果文件

当前可重点查看以下文件：

文件	说明
<code>outputs/zerogs_train/visuals/epoch_0412.png</code>	最新 epoch 可视化
<code>outputs/zerogs_train/plots/loss_curve.png</code>	loss 曲线
<code>outputs/zerogs_train/checkpoints/best.pt</code>	当前最佳验证 loss checkpoint
<code>outputs/zerogs_train/checkpoints/latest.pt</code>	最近保存的 latest checkpoint
<code>outputs/zerogs_train/stats/gaussian_stats_epoch_0412.json</code>	最新 Gaussian 统计
<code>outputs/zerogs_train/stats/performance_latest.json</code>	最新性能统计

6. 成果图



第一行为gt，第二行为模型输出的 Gaussian 重建后的结果，第三行为 gt mask，第四行为 Gaussian mask，第五行为 gt - Gaussian

6. 小结

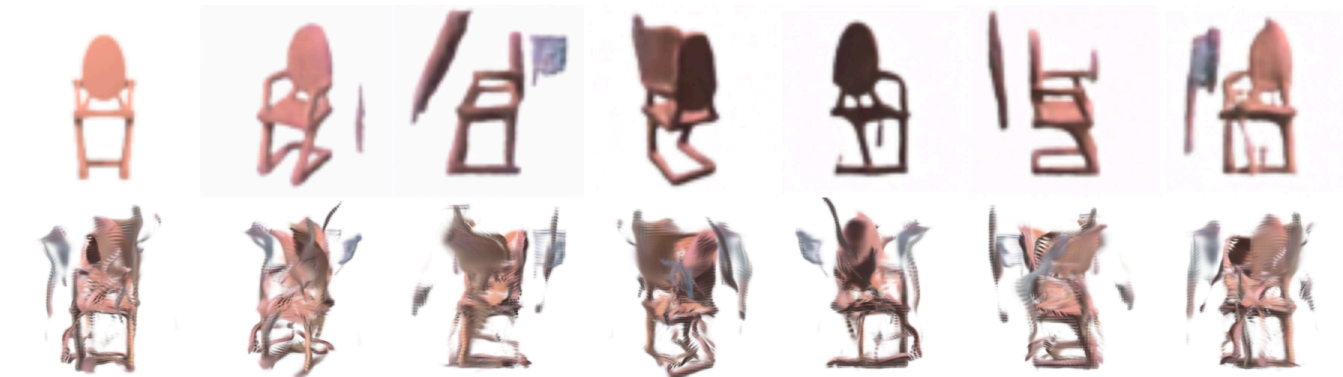
当前训练链路已经能够完成多视角输入、Gaussian 参数预测、`gsp1at` 渲染、重建损失计算、验证、可视化和性能统计。到 epoch 412 时，验证集 mean loss 为 `0.049461`，mean PSNR 为 `25.6270 dB`。

7. 关于两部分融合的管线

由于单图生多图的mini123部分受时间和进度限制，只使用了 chair 一个类，并且分辨率只采用了128*128，且目前实验性模型未能生成较好的多视角一致的多图。而 Gaussian 重建极其依赖

图像输入的质量，在低质量图像控制下效果会急剧恶化（直观上，模糊的质量差照片难以重建出好的 3D），故很遗憾我们现阶段无法使整个管线可用。

我编写了fuse部分的代码尝试过融合，但是结果比较差。



ps:

这个项目是真难做啊！开学初选定课题时没想那么多。前后 3D 相关的原理就学了好久。而且环境也配了好久。实验还经常出bug，好不容易跑起来了，发现显存占用很大，算力不够，优化的很慢。加上期末周，时间很急很急。

开学时野心很大，想要接轨一些看过的前沿论文。但实际开始做时已经快期末了，ddl就在眼前，只能把要求一降再降。

但有一说一，最后效果还行，算是一点点宽慰吧。